

HelixQA

HelixQA

Orchestration anti-bluff de QA — sessions autonomes multiplateformes où chaque SUCCÈS s'accompagne de preuves tangibles qu'un utilisateur réel peut exploiter une fonctionnalité.

Résumé

HelixQA est un cadre d'orchestration anti-bluff pour les tests multiplateformes (Android, Android TV, Web, Desktop) qui combine les banques de tests YAML, la détection en temps réel des plantages, la capture étape par étape des preuves, et des sessions de QA autonomes LLM enrichies par la vision par ordinateur pour démontrer que les fonctionnalités fonctionnent véritablement de bout en bout. Il s'agit du type de test QA imposé par Constitution (§11.4.169).

Description courte

Un orchestrateur anti-bluff de QA (Go) exécutant des banques de tests écrits et des sessions de QA entièrement autonomes, pilotées par LLM et la vision par ordinateur, sur l'ensemble des plateformes — détectant les plantages, validant chaque étape par rapport aux preuves capturées (captures d'écran, logcat, vidéo, traces de pile), et générant automatiquement des tickets riches en preuves pour les pipelines de correction AI.

Description détaillée

HelixQA est un cadre Go dont le principe de conception, intransigeant et unique, repose sur la Règle Opérationnelle §11.4 de Constitution : le critère de livraison n'est pas « les tests passent », mais « les utilisateurs peuvent utiliser la fonctionnalité ». Ainsi, chaque SUCCÈS qu'il émet doit s'appuyer sur des preuves tangibles capturées en cours d'exécution — sans

preuve, pas de validation, aucune exception. Il fonctionne selon deux modes complémentaires couvrant à la fois le scripté et l'inconnu.

Premièrement, les **banques de tests écrits** — des suites YAML composées de cas TC-XXX avec ciblage par plateforme, priorités, étapes ordonnées (nom/action/attendu), balises et références documentaires — exécutées avec validation à chaque étape, détection en temps réel des plantages et des ANR (ADB pour Android, surveillance des processus pour le web et le desktop), collecte centralisée des preuves, et génération automatique de tickets en Markdown déjà formatés pour les pipelines de correction AI en aval.

Deuxièmement, une **session de QA entièrement autonome** qui confie l'application à des agents alimentés par LLM et la vision par ordinateur, leur permettant de la piloter sans intervention à travers quatre phases disciplinées : la configuration (sélection des LLM, construction d'une cartographie des fonctionnalités à partir de la documentation du projet, déploiement des agents CLI, initialisation du moteur de vision), la vérification guidée par la documentation qui parcourt chaque fonctionnalité documentée, l'exploration par curiosité visant délibérément les cas limites et les comportements non documentés, puis la génération de rapports et le nettoyage au format Markdown/HTML/JSON, chaque constat étant lié à des preuves horodatées en vidéo.

Fondamentalement, il ne s'auto-évalue pas : il intègre quatre sous-modules externes Go (LLMsVerifier, LLMOrchestrator, VisionEngine, DocProcessor) et réutilise l'infrastructure partagée challenges et containers, de sorte que le composant qui navigue dans l'application n'est pas celui qui juge de son bon fonctionnement. Sa propre suite est soumise au même niveau d'exigence qu'il impose aux autres via make anti-bluff (analyse statique + manifeste d'ancrage comportemental + cliquet de mutation) et un Challenge d'orchestrateur en 8 phases doté d'une mutation intégrée (§1.1). Une matrice de couverture des types de tests sur 15 lignes rattache chaque capacité annoncée à un actif exécutable concret et à une forme spécifique de preuve capturée — ainsi, les affirmations du cadre sur lui-même sont aussi soumises à des preuves que les verdicts qu'il rend sur les produits testés.

Contenu

Pourquoi nous l'avons conçu

Les QA classiques donnent leur feu vert dès qu'« l'assertion est validée », ce qui laisse précisément passer la catégorie d'échecs que le Constitution qualifie de *bluff* - une fonctionnalité déclarée opérationnelle alors qu'elle est défaillante pour l'utilisateur réel.

Le HelixQA a été développé pour rendre cela impossible, spécifiquement pour les QA : il refuse d'attribuer un PASS sans preuve tangible (capture d'écran, logcat, vidéo, trace de pile, rapport) capturée en conditions réelles d'exécution, et considère une ligne de synthèse verte sans preuve comme un défaut critique équivalent à une fonctionnalité manquante. Il résout aussi le problème de charge de travail – les QA manuels exhaustifs sur de multiples plateformes ne sont pas scalables – en rendant les sessions entièrement autonomes.

Pourquoi c'est un changement de paradigme

Il fusionne deux éléments qui coexistent rarement dans un même outil : un contrôle qualité rigoureux, étayé par des preuves, et une exploration autonome et autoguidée. Un agent LLM couplé à la vision ouvre *l'application réelle*, vérifie chaque fonctionnalité documentée, traque les bugs non documentés pour lesquels personne n'a écrit de test, *et* produit une chaîne de preuves de qualité judiciaire – si bien que « nous l'avons testé » est remplacé par « voici la vidéo, voici le logcat, voici le ticket ». Et comme il s'agit du sous-module QA baptisé par le Constitution, son adoption ne se contente pas d'améliorer l'honnêteté des tests pour une seule équipe : elle relève le niveau minimal pour tous les produits de la famille en une seule opération.

Ce qui est innovant

- **Contrat de preuve anti-bluffer** – chaque validation de PASS est liée à une preuve d'exécution capturée ; une ligne verte en CI est considérée comme nécessaire mais jamais suffisante, et un résumé vert sans preuve est noté comme un défaut critique.
- **Exploration autonome guidée par la documentation et la curiosité** – il vérifie chaque fonctionnalité documentée *puis* sort des sentiers battus, testant les cas limites que rencontrent les utilisateurs réels (entrées vides, interactions rapides, chemins non documentés) et que les suites de tests manuels n'avaient pas anticipés.
- **Oracle visuel** – la vision mécanique GoCV associée au LLM Vision API *voit* littéralement l'interface utilisateur à l'écran, détectant les états visuellement defectueux que les assertions au niveau des tokens ou des propriétés ignorent.
- **Banques de tests structurées, non textuelles** – les chaînes des banques décrivent une structure et génèrent des questions dynamiques via LLM à l'exécution (CONST-046), permettant à une seule banque de fonctionner dans toutes les langues au lieu de se briser dès que le texte de l'interface est traduit.

- **Tickets optimisés pour les pipelines de correction AI** – les problèmes en Markdown générés automatiquement arrivent avec l'intégralité du dossier de preuves joint, prêts à être transmis directement à un agent de réparation en aval plutôt qu'à un humain chargé du tri.

Comment il est utilisé dans tous les produits (les pouvoirs qu'il confère)

En tant que **pilier qualité obligatoire** (le Constitution §11.4.169 désigne le sous-module `helix_qa` comme l'un des types de tests requis), le HelixQA dote chaque produit de la famille des mêmes capacités :

- **Sessions de QA autonomes** : une simple commande `helixqa autonomous --project ... --platforms android,desktop,web` lance un agent LLM couplé à la vision, qui pilote les applications réelles sans surveillance pour atteindre un objectif de couverture, générant rapports, tickets et vidéos sans intervention humaine.
- **Banques et suites de tests** : les banques YAML (niveau plancher 30 pour la version 219), ciblées par plateforme, classées par priorité et traçables ligne par ligne jusqu'à la documentation qu'elles vérifient.
- **Preuves capturées** : captures d'écran, logcat, vidéos, traces de pile et chronologie complète – centralisées et liées à chaque rapport, permettant de rejouer et d'auditer toute décision après coup.
- **Verdicts indépendants (§11.4.141 principe d'indépendance)** : son `issuetelector` alimenté par LLM et son oracle visuel évaluent le comportement de l'application en cours d'exécution indépendamment de l'agent qui l'a pilotée, éliminant structurellement le biais classique où un système valide son propre travail.
- **Barrière et cliquet de mutation** : les commandes `make qa-all / make anti-bluff et challenges/scripts/helixqa_orchestrator_challenge.sh` (8 phases, mutation intégrée §1.1) maintiennent en permanence la fiabilité du HelixQA – et il n'existe délibérément aucune échappatoire `--skip-helixqa` pour désactiver cette discipline sous la pression des délais.

Contenu

Principaux défis techniques et leurs solutions

- **Éviter les faux positifs dans le QA lui-même** — l'outil qui détecte les impostures ne doit pas en devenir une → chaque étape est validée par rapport à des preuves capturées, un

PASS sans preuve est considéré comme un défaut plutôt qu'un succès, et un manifeste d'ancrage comportemental lie chaque capacité annoncée à un test exécutable (CONST-035), de sorte qu'aucune capacité ne peut être revendiquée sans un élément qui l'exerce.

- **Piloter des plateformes hétérogènes depuis un seul cerveau** — Android, Android TV, Web et Desktop ne partagent aucun modèle d'entrée → un seul package navigator abstrait les ActionExecutors spécifiques à chaque plateforme (ADB, Playwright, X11) et les détecteurs de plantages par plateforme (Android/Web/Desktop), ce qui permet d'écrire la logique d'orchestration une seule fois et de cantonner les différences entre plateformes aux marges.
- **Rendre les agents autonomes utiles, et non chaotiques** — un LLM non supervisé lâché dans une application peut errer indéfiniment → LLMsVerifier évalue et sélectionne les bons modèles, LLMOrchestrator gère les agents CLI en mode headless (opencode, claude-code, gemini, junie, qwen-code), DocProcessor construit la carte des fonctionnalités qui donne une cible à l'exploration, et VisionEngine ancre chaque décision dans les pixels réels à l'écran plutôt que dans l'imagination du modèle.
- **Banques de tests sécurisées pour la localisation** — une suite qui intègre en dur du texte d'interface en anglais plante dans quinze langues → les banques décrivent uniquement la structure, et le texte des invites présenté à l'utilisateur est chargé dynamiquement via LLM/ressources au moment de l'exécution (CONST-046), si bien que la même banque vérifie le même comportement quelle que soit la langue.
- **Prouver que les garde-fous ne sont pas des leurres** — un garde-fou anti-imposture qui ne peut pas lui-même échouer est l'imposture ultime → des mutations appariées au §1.1 suppriment la capture de preuves ou l'assertion anti-imposture d'un type et exigent que le garde-fou ÉCHOUE, tandis qu'un cliquet de mutation empêche cette garantie de s'éroder silencieusement avec le temps.

Pile technologique

- **Orchestrateur Go 1.24+** — *pourquoi* : le QA doit pouvoir s'exécuter partout où les produits le font, donc un binaire unique, statiquement lié, rapide et portable l'emporte sur une alternative lourde en runtime ; *comment* : un seul binaire cmd/helixqa CLI exposant des sous-commandes composables run / list / report / autonomous / version.
- **Banques de tests YAML (pkg/testbank)** — *pourquoi* : les suites doivent être déclaratives et lisibles, modifiables par des humains sans toucher à Go ; *comment* : version/name/test_cases[] avec id, category, priority, platforms, des steps[]

ordonnés et `documentation_refs[]` pour assurer la traçabilité vers la documentation des fonctionnalités.

- **Détecteurs de plantages/ANR (pkg/detector)** — *pourquoi* : les échecs les plus critiques sont ceux qui surviennent en direct, pendant une interaction, et non dans une assertion a posteriori ; *comment* : ADB (pidof/logcat/screencap) pour Android et pgrep pour le Web/Desktop, surveillant le processus pendant que le test le pilote.
- **Collecte de preuves (pkg/evidence, pkg/session)** — *pourquoi* : le contrat anti-imposture n'est réel que si chaque PASS est étayé par une preuve tangible ; *comment* : captures d'écran, logcat, vidéos et traces de pile enregistrées dans une timeline SessionRecorder à laquelle chaque rapport renvoie.
- **Session autonome (pkg/autonomous, pkg/navigator, pkg/issuedetector)** — *pourquoi* : un QA manuel exhaustif sur quatre plateformes ne passe pas à l'échelle, donc l'exploration doit être autoguidée ; *comment* : un SessionCoordinator en 4 phases, associé à des ActionExecutors (ADB/Playwright/X11) et à une détection de bugs LLM couvrant les défauts visuels, UX, d'accessibilité et fonctionnels.
- **Sous-modules externes** — *pourquoi* : réutilisation et découplage (CONST-051), et — point crucial — séparation entre le navigateur et l'arbitre ; *comment* : LLMsVerifier (évaluation des modèles), LLMOrchestrator (agents CLI en mode headless), VisionEngine (GoCV + Vision LLM), DocProcessor (carte des fonctionnalités/couverture), chacun étant un composant indépendant.
- **Garde-fous anti-imposture + cliquet de mutation** — *pourquoi* : pour soumettre HelixQA au même pacte §1.1 qu'il impose à tout le reste ; *comment* : un scan make anti-bluff couplé à un manifeste d'ancrage comportemental et à un cliquet de mutation, avec `helixqa_orchestrator_challenge.sh` comme validateur de bout en bout en 8 phases.
- **Matrice de couverture à 15 lignes (docs/test-coverage.md)** — *pourquoi* : CONST-050(B) impose un ensemble de types de tests fermé et entièrement documenté, sans lacunes ; *comment* : chaque ligne est liée à un actif exécutable concret et à une forme spécifique de preuve capturée, de sorte que la couverture est un fait vérifié plutôt qu'une simple affirmation.

Contenu

Notes sur le statut et l'honnêteté

- **Statut : bêta.** En développement actif (bannière de statut du README, version 219). Soumis à sa propre norme anti-bidon.
- **Licence : Apache-2.0.** Installation : `go install digital.vasic.helixqa/cmd/helixqa@latest`.

Niveau de priorité : Helix-primaire — un pilier obligatoire de qualité/anti-bidon pour la manière dont la famille Helix vérifie que les fonctionnalités fonctionnent réellement.